



MÓDULO 2

Fundamentos mínimos para dirigir a Codex

Cómo entender lo suficiente del código para pedir mejor, corregir mejor y validar mejor en Isaval

Propósito del capítulo

Este módulo no pretende convertir al alumno en programador. Su propósito es mucho más práctico: darle el vocabulario mínimo para que deje de ver el código como una caja negra y pueda trabajar con Codex con menos miedo, más precisión y mayor capacidad de revisión.

Resultado esperado	Poder leer un script sencillo, entender su lógica básica y pedir ajustes pequeños con criterio.
Aplicación inmediata	Mejorar prompts, corregir reglas simples y validar cambios generados por Codex.

1. El objetivo real de este módulo

Una de las grandes barreras en usuarios no técnicos no es la instalación, sino la sensación de extrañeza ante el código. Cuando ven unas cuantas líneas con variables, paréntesis y bloques, muchos llegan a la misma conclusión demasiado pronto: 'esto no es para mí'. Ese pensamiento es comprensible, pero es falso. Para usar bien Codex no hace falta dominar la programación en sentido clásico. Hace falta entender lo suficiente para saber qué se está haciendo, qué se está cambiando y qué se debería comprobar.

El alumno de Isaval no necesita convertirse en desarrollador. Necesita adquirir una alfabetización funcional. Eso significa saber distinguir dónde entra la información, dónde se decide algo, dónde se repite una acción y dónde aparece la salida final. Con ese nivel de comprensión ya puede dialogar mucho mejor con Codex y, sobre todo, puede detectar cuándo una propuesta le resulta demasiado amplia, ambigua o arriesgada.

Este módulo enseña exactamente esa capa de comprensión mínima. Se trabajará con cuatro ideas básicas: variables, condiciones, bucles y funciones. Son palabras técnicas, sí, pero aquí no se estudiarán como teoría abstracta. Se explicarán como piezas de trabajo que permiten entender scripts reales y pedir cambios sensatos.

Traducción práctica

No se trata de aprender a escribir mucho código. Se trata de aprender a leer lo suficiente como para no depender ciegamente de lo que Codex proponga.

2. Qué es una variable

La forma más simple de entender una variable es pensar en ella como una etiqueta que guarda un dato. Ese dato puede ser un nombre, una ruta, una fecha, una cantidad o cualquier otra información que el script necesite usar. Si se quiere una comparación empresarial, una variable se parece mucho a una celda importante de Excel que después se reutiliza varias veces.

Cuando Codex genera un script para trabajar con carpetas, es muy habitual que use variables como ``carpeta_origen``, ``archivo_salida``, ``clientes``, ``fecha_actual`` o ``prefijo``. El alumno no tiene que memorizar sintaxis exactas. Lo importante es que aprenda a mirar un bloque y preguntarse: qué datos se están guardando aquí y para qué se reutilizarán después.

<code>`carpeta_origen`</code>	Lugar desde el que el script toma los archivos.
<code>`nombre_nuevo`</code>	Texto que se usará para renombrar o generar una salida.
<code>`fecha`</code>	Dato temporal que puede insertarse en nombres o informes.
<code>`lista_clientes`</code>	Conjunto de elementos sobre los que el script trabajará después.

Qué debe retener el alumno

Cuando vea una variable, no debe pensar 'esto es programación avanzada'. Debe pensar: aquí están guardando un dato importante que luego se reutilizará.

3. Qué es una condición

Una condición es una decisión. Dicho de la forma más cotidiana posible: si ocurre A, entonces se hace B; si no, se hace otra cosa o no se hace nada. Las condiciones aparecen constantemente en scripts generados por Codex porque casi todos los trabajos reales requieren distinguir casos normales, casos especiales y elementos que conviene excluir.

En Isaval, una condición puede servir para no procesar archivos que contengan la palabra `BORRADOR`, para ignorar fotos demasiado pequeñas, para separar presupuestos de facturas o para aplicar una regla distinta según el tipo de documento. Entender esto cambia mucho la relación con el código, porque el alumno empieza a reconocer que una gran parte de los ajustes no consiste en rehacer el script entero, sino en cambiar una regla de decisión.

Si el archivo contiene `BORRADOR`	No se procesa.
Si la extensión es `.pdf`	Se trata como documento final.
Si el importe es cero	Se marca para revisión.
Si el nombre está vacío	Se registra incidencia.

La enseñanza clave de esta sección es muy útil para el resto del curso: muchos cambios que parecen complejos son, en realidad, pequeños ajustes de condición. Si el alumno aprende a localizar dónde se toma una decisión, empezará a pedir modificaciones mucho más precisas a Codex.

4. Qué es un bucle

Un bucle es una repetición organizada. Sirve para aplicar la misma acción sobre varios elementos sin tener que escribir la instrucción una y otra vez. Si un script revisa 300 archivos, 200 filas de Excel o 50 clientes, es muy probable que haya un bucle recorriendo esa lista de elementos.

Para el alumno, la clave no es memorizar la forma exacta del bucle, sino reconocer su función. Cuando vea una estructura repetitiva, debe pensar: aquí están pasando por todos los elementos de un conjunto. Esa lectura funcional es suficiente para entender por qué el script afecta a muchos archivos, muchas filas o muchos registros.

También es importante desde el punto de vista del riesgo. Un bucle mal entendido puede ser la razón por la que una acción aparentemente pequeña termine afectando a cientos de elementos. Por eso, cuando el alumno vea que el código itera sobre una lista o una carpeta completa, debe aumentar su atención y exigir pruebas sobre muestras pequeñas antes de escalar.

Lectura rápida de un bucle

Si el script pasa por una lista, por una carpeta o por una tabla fila a fila, probablemente está usando un bucle. Eso significa repetición y, por tanto, potencial de impacto masivo si la regla es incorrecta.

5. Qué es una función

Una función es un bloque de trabajo con una tarea concreta. Sirve para ordenar el código y reutilizar la misma lógica varias veces sin copiarla entera. Si una variable se parece a una celda de Excel y una condición a una decisión, una función puede compararse con una pequeña herramienta o mini proceso dentro de una cadena mayor.

Por ejemplo, una función podría servir para limpiar nombres de archivo, otra para generar un informe y otra para detectar duplicados. El alumno no necesita aprender a diseñarlas desde cero, pero sí le conviene reconocer que cuando Codex agrupa varias líneas bajo un nombre de función, en realidad está empaquetando una tarea específica.

<code>`limpiar_nombre()`</code>	Normalizar títulos de archivos o productos.
<code>`detectar_duplicados()`</code>	Revisar documentos repetidos en una carpeta.
<code>`generar_resumen()`</code>	Preparar una salida clara para dirección o comercial.

El valor práctico de entender las funciones es alto. Ayuda a leer mejor bloques largos, a pedir a Codex explicaciones más ordenadas y a localizar con rapidez qué parte del script conviene revisar cuando algo no se comporta como se esperaba.

6. Cómo leer un script sin asustarse

La mejor manera de leer un script sencillo no es línea por línea. Esa estrategia suele fatigar a quien no tiene hábito técnico. La forma correcta para este curso es leer por bloques funcionales. Primero se identifica la entrada, luego la lógica y finalmente la salida. Con ese marco, el texto deja de parecer un muro y empieza a convertirse en una secuencia entendible.

- Entrada: qué archivos, datos o carpetas recibe el script.
- Lógica: qué decisiones toma, qué repite y qué transforma.
- Salida: qué nuevo archivo, renombrado, resumen o resultado produce.

Esta forma de lectura debe convertirse en una costumbre del alumno. Cuando pida a Codex que le explique un script, debería exigir casi siempre esa estructura: 'explícamelo como entrada, lógica y salida'. Esa sola petición mejora mucho la comprensión y reduce la sensación de oscuridad técnica.

Prompt útil para entender un script

Explícame este script como si yo fuera un usuario de negocio.

No me lo cuentes línea a línea.

Sepáralo en tres partes: entrada, lógica y salida.

Después indícame qué bloque tendría que revisar si quisiera cambiar una regla concreta.

7. Caso guiado: excluir borradores en una carpeta comercial

Supongamos una carpeta de trabajo del área comercial con presupuestos, documentos finales y varias versiones internas identificadas con palabras como `BORRADOR`, `PENDIENTE` o `REVISION`. Se desea usar Codex para preparar una automatización que ordene o procese esos archivos, pero evitando que los borradores contaminen el resultado final.

Este caso es muy útil para el módulo porque muestra cómo una pequeña regla de programación puede cambiar el comportamiento completo del sistema. La clave no está en reescribirlo todo, sino en localizar la condición adecuada. El alumno debe empezar a ver que muchas mejoras prácticas consisten en tocar con precisión una decisión concreta y no en pedir una nueva solución desde cero.

Variable	Dónde se guarda el nombre del archivo.
Condición	Dónde se decide excluir `BORRADOR`.
Bucle	Dónde se repite la revisión para todos los archivos.
Salida	Qué resultado final se genera o qué archivos se procesan.

Prompt de trabajo sobre el caso

Analiza este script y explícamelo como entrada, lógica y salida.

Después localiza la parte donde debería introducirse una regla para no procesar archivos cuyo nombre contenga la palabra BORRADOR.

No rehagas el script entero; señala solo el bloque o línea relevante y explica por qué.

8. Qué debe responder Codex en un caso como este

Una buena respuesta de Codex no debería limitarse a devolver una versión nueva del script sin contexto. Lo correcto es que primero explique el flujo general y luego señale con claridad dónde se encuentra la decisión que filtra o debería filtrar los archivos. Si el alumno no recibe esa explicación, conviene pedírsela. El propósito de este módulo no es producir cambios rápidos, sino comprensión útil.

Una respuesta sólida diría, por ejemplo, que el script toma archivos de una carpeta, recorre cada nombre mediante un bucle, evalúa ciertas condiciones y genera una salida posterior. Después señalaría que el sitio correcto para excluir borradores es el bloque donde se comprueba el nombre del archivo. Ese tipo de explicación es precisamente lo que entrena al alumno para interactuar con Codex con más precisión.

Qué se busca en la explicación

Que Codex señale el lugar exacto donde vive la regla, y que explique el cambio en lenguaje simple antes o junto a la modificación técnica.

9. Práctica guiada

La práctica de este módulo debe ser deliberadamente pequeña. El objetivo no es escribir un programa nuevo desde cero, sino aprender a leer uno sencillo, identificar su estructura y pedir un ajuste localizado. En una formación para Isaval esto es mucho más valioso que una falsa sensación de avance basada en copiar y pegar sin comprensión.

- Tomar un script breve generado por Codex para trabajar con archivos o datos.
- Pedir explicación por bloques: entrada, lógica y salida.
- Identificar al menos una variable, una condición, un bucle y una función.
- Solicitar un pequeño ajuste: por ejemplo, excluir `BORRADOR` o ignorar archivos vacíos.
- Comparar el comportamiento antes y después del cambio.
- Redactar una nota final explicando qué parte de la lógica se ha modificado.

Resultado esperado de la práctica

El alumno no debe salir diciendo 'ya sé programar'. Debe salir diciendo: 'ya no veo el script como una caja negra; sé dónde mirar y qué pedir'.

10. Errores típicos en esta fase

Cuando un usuario empieza a familiarizarse con el código, puede caer en dos errores opuestos. El primero es pensar que sigue sin entender nada y retirarse demasiado pronto. El segundo es creer que ya puede modificarlo todo sin ayuda y tocar más de la cuenta. Ninguno de los dos extremos conviene.

Leer línea por línea y bloquearse	Falta de método de lectura	Volver a entrada, lógica y salida.
Pedir a Codex reescrituras enteras	Falta de precisión	Pedir cambios localizados.
Tocar varias reglas a la vez	Pérdida de trazabilidad	Cambiar una sola cosa y volver a probar.
Aceptar el cambio sin comparar	Validación débil	Revisar el antes y el después.

El criterio operativo del módulo es simple: una modificación pequeña, bien entendida y bien validada vale mucho más que una reescritura grande poco comprendida. Esa mentalidad prepara muy bien al alumno para los siguientes bloques del curso.

11. Aplicaciones directas en Isaval

Este módulo tiene una transferencia muy clara a la vida real de la empresa. Cuanto mejor entienda el alumno la lógica mínima de los scripts, mejor podrá pedir a Codex pequeñas adaptaciones de valor inmediato. Esto reduce dependencia, mejora la calidad de los prompts y acorta mucho el tiempo necesario para corregir una automatización.

Comercial	Excluir borradores, filtrar listados y ajustar reglas de clasificación.
Administración	Modificar pequeñas condiciones en consolidaciones o limpiezas de datos.
Técnica	Entender scripts de renombrado o clasificación documental sin rehacerlos completos.
Dirección	Pedir mejor explicaciones y validar si un cambio tiene sentido de negocio.

La mejora real de este módulo no es visual ni teórica. Es una mejora de control. El usuario empieza a ganar precisión. Y en un trabajo con IA aplicada a procesos, la precisión vale muchísimo.

12. Examen de evolución del módulo

La evaluación debe comprobar comprensión funcional, no memoria de sintaxis. Lo importante es que el alumno demuestre que entiende qué piezas hay dentro de un script, cómo leerlo con método y cómo pedir ajustes pequeños con criterio.

Instrucciones

Duración orientativa: 25 a 30 minutos. Puntuación recomendada: 20 puntos. Se aconseja exigir al menos 14 puntos para considerar superado el módulo.

Pregunta 1 · 2 puntos

¿Qué objetivo refleja mejor este módulo?

- A. Aprender varios lenguajes de programación.
- B. Entender lo suficiente del código para dirigir y revisar mejor a Codex.
- C. Sustituir completamente a un equipo técnico.
- D. Memorizar sintaxis compleja.

Pregunta 2 · 2 puntos

¿Qué es una variable en lenguaje simple?

- A. Un error del sistema.
- B. Una etiqueta que guarda un dato reutilizable.
- C. Una carpeta temporal.
- D. Una clase de permiso.

Pregunta 3 · 2 puntos

¿Qué describe mejor una condición?

- A. Una decisión del tipo si ocurre A, se hace B.
- B. Un diseño visual del programa.

C. Una copia de seguridad.

D. Un archivo de salida.

Pregunta 4 · 4 puntos

Explica en pocas líneas para qué sirve un bucle y por qué merece atención desde el punto de vista del riesgo.

Pregunta 5 · 4 puntos

Describe cómo leerías un script sencillo sin analizarlo línea a línea.

Pregunta 6 · 4 puntos

Escribe un prompt útil para pedir a Codex que localice la parte de un script donde excluir archivos con la palabra BORRADOR.

Pregunta 7 · 2 puntos

Resume en una frase qué aporta entender una función a un usuario no técnico.

13. Clave de corrección

1	B	Debe quedar claro que el objetivo es comprensión funcional, no programación académica.
2	B	Se espera la idea de dato guardado y reutilizable.
3	A	La condición debe entenderse como decisión.
4	Respuesta abierta	Debe mencionar repetición organizada y riesgo de impacto masivo si la regla es errónea.
5	Respuesta abierta	Se espera lectura por entrada, lógica y salida.
6	Respuesta abierta	Debe pedir explicación y localización precisa del bloque de decisión, no reescritura completa.
7	Respuesta abierta	Debe reflejar que una función agrupa una tarea concreta y ayuda a ordenar la lectura.

Cierre del módulo

Al terminar este capítulo, el alumno debería sentir algo muy concreto: que ya no está completamente fuera del código. No necesita dominarlo todo. Le basta con haber dejado de verlo como una pared opaca. Ese cambio de percepción es un salto enorme para trabajar mejor con Codex en Isaval.